

Logger

Table of Contents

- [Overview](#)
 - [Design](#)
 - [The Observer Pattern](#)
 - [Emitter - Consumer Pairs](#)
 - [Emitter](#)
 - [Consumer](#)
 - [Log Filter](#)
 - [Log Format](#)
-

Overview

This module is an abstraction of the logging functionality, allowing setting up a logger server to which clients can send logging entries.

Logger library supports the following features:

- Different clients can add new entries to the log asynchronously without corrupting the output.
- Logs can be printed on the console (stdout).
- Logs can be printed to the file.
- Log level filter can be configured both on the client and server-side.
- Log entry can be of a max of the page size.
- Each client has a unique ID which is appended to the log entry, and optionally a name.
- Custom logging format can be added.

Design

Due to the mentioned above features, the following design patterns have been used in this library.

The Observer Pattern

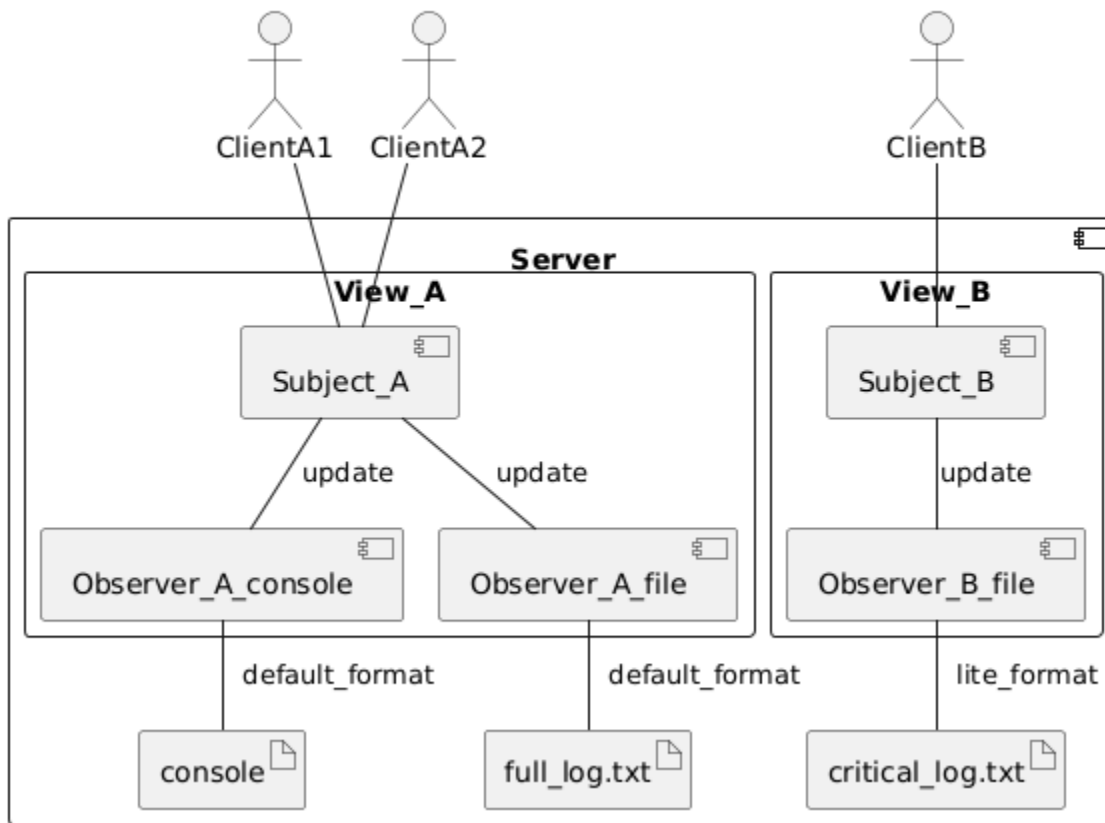
Log server contains one or more Views i.e. a collection of Subjects to which Observers are connected.

The typical example would be that Log Server has a "ViewA" (*SubjectA* with two Observers) with the entries being displayed in the default format (@see *OSLoggerFormat.h*) and being print on the console and *fulllog.txt* (i.e. the two Observers are console and *fulllog.txt*), and a *ViewB* with the entries being displayed in the custom format (e.g. lite one with less data) stored in the file *critical_log.txt*.

Log clients are now mapped to the proper views based on the requirements.

See the below diagram for a better understanding:

Logger Component Diagram



Emitter - Consumer Pairs

The Client-Server model is implemented by the introduction of log entries emitters and consumers.

Emitter

If Emitter wants to log a new entry, it copies the data to the exchange buffer (Client-Server shared memory) and it does the RPC call `emit`.

Consumer

On the server-side, there exists a list of consumers. Each consumer is assigned to one client (1 to 1 consumer-emitter pair).

When `emit` was called, Server iterates over the consumer list and finds the one that corresponds to the emitter. Chosen consumers process and forwards the log entry to the corresponding Subject.

Log Filter

Both on the client and server-side it is possible to configure a filter for the log level.

If entry's level is greater then the configured filtering level, it will be dropped without further processing.

Please note that client-side filtering is done earlier before entry data is copied, so it is more efficient.

Log Format

When the entry is about to be copied to the target directory, it can be formatted in the given way.

This can be done by overriding `OS_LoggerAbstractFormat_vtable_t::convert` function.